



Instituto Politécnico Nacional

Escuela Superior de Cómputo

ESCOM

Proyecto final

Sistema de búsqueda y recomendación de libros

Fecha de entrega: 4 de abril de 2025

Ingeniería de software

Grupo: 6CV3

Presenta:

Hernández Jiménez Irmin





Índice

Tabla de versiones del documento	vi
Documento de requerimientos	7
Introducción	8
Objetivos	8
Alcance del sistema	8
Metodología	8
Visión general del sistema	9
Descripción general	9
Contexto del sistema.....	9
Características principales	9
Usuarios del sistema	9
Usuario regular.....	9
Administrador	10
Requerimientos funcionales.....	11
Requerimientos no funcionales.....	16
Casos de uso	18
Diagramas de casos de uso	19
Caso de uso principal	29
Guía técnica	30
Herramientas utilizadas	31
Backend	31
Frontend.....	31
Base de datos	32
DevOps	32



Instituto Politécnico Nacional
Escuela Superior de Cómputo



Control de versiones	32
Configuración de la base de datos	33
Configuración de JPA/Hibernate	33
Configuración del sistema de seguridad (Spring Security)	34
Servicio de detalles de usuario	34
Unidades de la gestión JWT	35
Configuraciones de Seguridad.	35
Roles, permisos y endpoints protegidos.....	35
Roles:.....	35
EndPoints:.....	35
Permisos	40
Archivos relacionados.....	41
application.properties	41
docker-compose.yml	42
build.gradle.....	42
SecurityConfig.java	42
JwtUtil.java	43
CORS (Cross-Origin Resource Sharing).....	44
Variables de Entorno	44
Apéndices.....	46
Apéndice A: Glosario de términos.....	47



Índice de figuras

Figura 1. Diagrama de casos de uso de autenticación.....	19
Figura 2. Diagrama de casos de uso para la búsqueda y visualización de libros..	21
Figura 3. Diagrama de casos de uso para las acciones de un usuario autenticado.	24
Figura 4. Diagrama de casos de uso para un administrador.	27



Índice de tablas

Tabla 1. Tabla de especificación de casos de uso para el inicio de sesión en el sistema.....	20
Tabla 2. Tabla de especificación de casos de uso para el registro en el sistema. .	21
Tabla 3. Especificación del caso de uso para la búsqueda de un libro.....	22
Tabla 4. Especificación del caso de uso para el filtrado de los libros.	23
Tabla 5. Especificación del caso de uso para visualizar un libro en específico.	23
Tabla 6. Especificación de caso de uso para gestionar los temas de interés de un usuario.	25
Tabla 7. Especificación de caso de uso para la visualización del perfil de un usuario.	25
Tabla 8. Especificación del caso de uso para la recomendación de libros.	26
Tabla 9. Especificación de caso de uso para la administración de usuarios	28
Tabla 10. Especificación de caso de uso para la visualización de estadísticas.....	29



Tabla de versiones del documento

Versión	Cambio	Fecha	Descripción
1.0	Versión inicial del documento.	05/03/2025	En esta primera versión se agregaron los puntos fundamentales del documento.
1.1	Adición de los casos de uso	01/04/2025	Se añadieron secciones de glosario, referencias, visión general, usuarios, restricciones, y suposiciones y dependencias.



Instituto Politécnico Nacional

Escuela Superior de Cómputo

Documento de requerimientos



ESCOM



Introducción

En este documento se abordan las funcionalidades principales con las que contará el sistema de recomendación de libros, dividiendo estos apartados en los dos aspectos principales, los requerimientos funcionales y los requerimientos no funcionales.

Objetivos

A continuación, se listan los objetivos que se buscan alcanzar con el desarrollo de este proyecto.

Objetivo General

Recomendar libros a los usuarios de acuerdo con sus hábitos de búsqueda e información que proporcione al sistema.

Objetivo Particular

Diseñar un sistema que permita la búsqueda y recomendación de libros a los usuarios de acuerdo con sus preferencias.

Alcance del sistema

El sistema utilizará la API de OpenLibrary, por lo que el sistema solo se encargará de procesar las solicitudes de una forma correcta y estructurada que permita a los usuarios buscar entre libros disponibles en la OpenLibrary y recomendaciones de acuerdo a los libros registrados en esta API.

Metodología

La metodología por utilizar para el desarrollo de este proyecto de software es scrum, una metodología de desarrollo ágil que permite trabajar el proyecto mediante incrementos, con un enfoque de entregas continuas, a diferencia de una metodología incremental, scrum enfoca los esfuerzos en el desarrollo, minimizando los esfuerzos de una documentación excesiva.



Visión general del sistema

En esta sección, se explica de forma general las principales funciones y características del sistema.

Descripción general

El sistema de búsqueda y recomendación de libros es un sistema que permite a sus usuarios buscar entre una amplia variedad de libros, así como recibir recomendaciones de acuerdo con sus preferencias.

Contexto del sistema

El sistema se integra en un ecosistema de análisis de información para:

- Hacer solicitudes a la API de OpenLibrary de acuerdo con las preferencias del usuario.
- Almacenar y procesar las preferencias de un usuario.

Características principales

- Búsqueda de libro.
- Filtrado de búsqueda.
- Manejo de preferencias de acuerdo con los géneros.
- Recomendación de libros con base en las preferencias del usuario.

Usuarios del sistema

A continuación, se presentan los usuarios con los que operará el sistema, explicando los roles y sus funciones.

Usuario regular

- **Características:** Usuarios que utilizan el sistema para buscar libros y solicitar recomendaciones.
- **Nivel de experiencia:** Familiaridad con interfaces web.



Instituto Politécnico Nacional
Escuela Superior de Cómputo



- **Frecuencia de uso:** Uso regular (semanal o diario).
- **Privilegios:** Buscar libros, manejar sus preferencias y solicitar recomendaciones.

Administrador

- **Características:** Usuarios con capacidades de gestión del sistema.
- **Nivel de experiencia:** Conocimiento avanzado en administración de sistemas y gestión de usuarios.
- **Frecuencia de uso:** Uso periódico para tareas de mantenimiento y gestión.
- **Privilegios:** Todos los privilegios de usuario regular, más capacidades de gestión de usuarios, configuración del sistema y monitoreo.



Requerimientos funcionales

ID: RF1

Nombre del Requerimiento: Registro de Usuarios.

Descripción: Permitir que los usuarios se registren con un nombre, correo electrónico y contraseña.

Usuario al que Aplica: Todos los usuarios.

Precondiciones: Ninguna.

Flujo Principal:

1. El usuario ingresa su nombre, correo electrónico y contraseña.
2. El sistema valida que el correo electrónico no esté registrado.
3. El sistema almacena los datos del usuario en la base de datos.
4. El usuario es redirigido a la página de inicio dentro del sistema.

Flujo Alternativo:

1. El usuario ingresa un correo electrónico ya registrado.
2. El sistema notifica al usuario que el correo ya está en uso.

Postcondiciones: El usuario queda registrado en el sistema y recibe un correo de confirmación.

ID: RF2

Nombre del Requerimiento: Inicio de sesión.

Descripción: Permitir que los usuarios inicien sesión con su correo electrónico y contraseña.

Usuario al que Aplica: Todos los usuarios.



Instituto Politécnico Nacional
Escuela Superior de Cómputo



Precondiciones: El usuario debe estar registrado.

Flujo Principal:

1. El usuario ingresa su correo electrónico y contraseña.
2. El sistema valida las credenciales del usuario.
3. El sistema genera un JWT y lo envía al usuario.
4. El usuario accede a su cuenta.

Flujo Alternativo:

1. El usuario ingresa credenciales incorrectas.
2. El sistema notifica al usuario que las credenciales son inválidas.

Postcondiciones: El usuario queda autenticado y recibe un JWT para la sesión.

ID: RF3

Nombre del Requerimiento: Visualización de Información del usuario.

Descripción: Permitir que los usuarios vean su propia información una vez hayan iniciado sesión.

Usuario al que Aplica: Todos los usuarios.

Precondiciones: El usuario debe estar autenticado.

Flujo Principal:

1. El usuario accede a su cuenta.
2. El sistema muestra la información del usuario.

Flujo Alternativo: Ninguno.

Postcondiciones: El usuario puede ver su información personal.



ID: RF4

Nombre del Requerimiento: Acceso al dashboard de administrador.

Descripción: Permitir que los usuarios con rol de Administrador accedan a un dashboard donde se listan todos los usuarios.

Usuario al que Aplica: Administrador.

Precondiciones: El usuario debe estar autenticado y tener el rol de Administrador

Flujo Principal:

1. El administrador accede a su cuenta.
2. El sistema verifica el rol del usuario.
3. El sistema muestra el dashboard con la lista de todos los usuarios.

Flujo Alternativo:

1. Un usuario sin rol de Administrador intenta acceder al dashboard.
2. El sistema deniega el acceso.

Postcondiciones: El administrador puede ver la lista de todos los usuarios.

ID: RF5

Nombre del Requerimiento: Actualización de Información de usuarios por administrador.

Descripción: Permitir que los administradores actualicen la información de otros usuarios.

Usuario al que Aplica: Administrador.

Precondiciones: El administrador debe estar autenticado y tener acceso al dashboard.

Flujo Principal:



Instituto Politécnico Nacional
Escuela Superior de Cómputo



1. El administrador selecciona un usuario en el dashboard.
2. El administrador actualiza la información del usuario.
3. El sistema valida y guarda los cambios.

Flujo Alternativo:

1. El correo al que se actualiza ya está en uso
2. El sistema rechaza la solicitud y no realiza ningún cambio

Postcondiciones: La información del usuario se actualiza en la base de datos.

ID: RF6

Nombre del Requerimiento: Eliminación de usuarios por un administrador.

Descripción: Permitir que los administradores eliminen a otros usuarios.

Usuario al que Aplica: Administrador.

Precondiciones: El administrador debe estar autenticado y tener acceso al dashboard.

Flujo Principal:

1. El administrador selecciona un usuario en el dashboard.
2. El administrador elimina al usuario.
3. El sistema remueve la información del usuario de la base de datos.

Flujo Alternativo: Ninguno.

Postcondiciones: El usuario es eliminado del sistema.

ID: RF7.

Nombre del Requerimiento: Búsqueda y filtrado de libros.



Instituto Politécnico Nacional
Escuela Superior de Cómputo



Descripción: Permitir a los usuarios buscar libros en el sistema y filtrar los resultados según varios criterios.

Usuario al que Aplica: Usuario visitante y usuario autenticado.

Precondiciones: Encontrarse en la página de inicio.

Flujo Principal:

1. El usuario ingresa términos de búsqueda en el campo de búsqueda de libros.
2. El sistema muestra los resultados que coinciden con los términos de búsqueda.
3. El usuario puede aplicar filtros (por ejemplo, por autor, género, año de publicación, etc.).
4. El sistema actualiza los resultados en tiempo real según los filtros aplicados.
5. El usuario selecciona un libro de los resultados para ver más detalles.

Flujo Alterno:

- Si no se encuentran libros que coincidan con la búsqueda, el sistema muestra un mensaje informando que no se encontraron resultados.
- Si no se aplican filtros, se muestran todos los resultados de la búsqueda sin restricciones adicionales.

Postcondiciones: El usuario visualiza una lista de libros filtrados y puede acceder a los detalles de cada uno.



Requerimientos no funcionales

ID: SEC-1

Nombre: Seguridad.

Descripción: El sistema debe utilizar JWT para el manejo de sesiones y validación de permisos de usuario para cada acción.

ID: CNC-1.

Nombre: Concurrencia.

Descripción: El sistema debe ser capaz de manejar al menos 1000 usuarios concurrentes en un día sin degradación significativa en el rendimiento (en las condiciones de hardware óptimas).

ID: ESC-1.

Nombre: Escalabilidad.

Descripción: El sistema debe ser escalable para soportar futuras expansiones y aumentos en la base de usuarios, utilizando un enfoque modular en el backend.

ID: CMP-1.

Nombre: Compatibilidad.

Descripción: El frontend debe ser compatible con los navegadores modernos y dispositivos móviles.

ID: MNT-1



Instituto Politécnico Nacional
Escuela Superior de Cómputo



Nombre: Mantenibilidad.

Descripción: El código del sistema debe estar bien documentado y seguir buenas prácticas de desarrollo para facilitar su mantenimiento y actualización.

ID: USB-1

Nombre: Usabilidad

Descripción: El sistema debe ser fácil de usar e intuitivo tanto para usuarios regulares como para administradores. Pudiendo acceder al 80% de las funcionalidades con menos de 5 clics a los hipervínculos o botones para acceder a los recursos.

ID: DSP-1

Nombre: Disponibilidad.

Descripción: El sistema debe tener una alta disponibilidad, con un tiempo de inactividad mínimo. Siendo así, que se cumpla una disponibilidad del 99% del tiempo.



Instituto Politécnico Nacional

Escuela Superior de Cómputo

Casos de uso



Diagramas de casos de uso

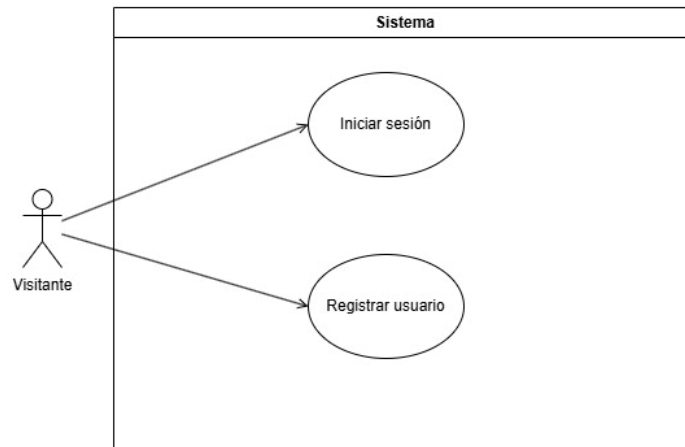


Figura 1. Diagrama de casos de uso de autenticación.

En la Figura 1 se muestra el diagrama de casos de uso referente a la autenticación de los usuarios, ya sea registro o inicio de sesión, en las tablas Tabla 1 y Tabla 2 se desglosa la especificación de los casos de uso.

ID	AUTH-1	
Nombre	Iniciar sesión	
Actores	Usuario no autenticado o visitante	
Descripción	Permite a un visitante ingresar al sistema proporcionando credenciales válidas.	
Precondición	- El visitante debe estar registrado en el sistema. - El sistema debe estar en funcionamiento.	
Flujo principal	Paso	Acción
	1.	El visitante ingresa su nombre de usuario y contraseña.
	2.	El sistema valida las credenciales.
	3.	Si las credenciales son correctas, el visitante accede al sistema como usuario autenticado.



Instituto Politécnico Nacional
Escuela Superior de Cómputo



Postcondiciones	El usuario queda registrado dentro del sistema, su dashboard actual es el inicio de este usuario además de recibir un correo de confirmación	
Flujo alterno	Paso	Acción
	1.	Si las credenciales son incorrectas, el sistema muestra un mensaje de error.
	2.	Se permite al visitante reintentar la entrada.
Comentarios:		

Tabla 1. Tabla de especificación de casos de uso para el inicio de sesión en el sistema.

ID	AUTH-2	
Nombre	Registrar usuario	
Actores	Usuario no autenticado o visitante	
Descripción	Permite a un visitante crear una cuenta en el sistema proporcionando la información requerida.	
Precondición	- El visitante no debe estar registrado previamente. - El sistema debe estar en funcionamiento.	
Flujo principal	Paso	Acción
	1.	El visitante accede a la opción de registro.
	2.	Ingresa la información requerida (nombre, correo electrónico y contraseña).
	3.	El sistema valida la información ingresada.
	4.	Si la información es válida, el sistema crea la cuenta y confirma el registro.
Postcondiciones	Si el registro es exitoso, el usuario puede autenticarse en el sistema.	
Flujo alterno	Paso	Acción
	1.	Si la información proporcionada es incorrecta, incompleta o ya está registrado el

		correo, el sistema muestra un mensaje de error.
	2.	Se permite al visitante corregir los datos y reintentar.
Comentarios:		

Tabla 2. Tabla de especificación de casos de uso para el registro en el sistema.

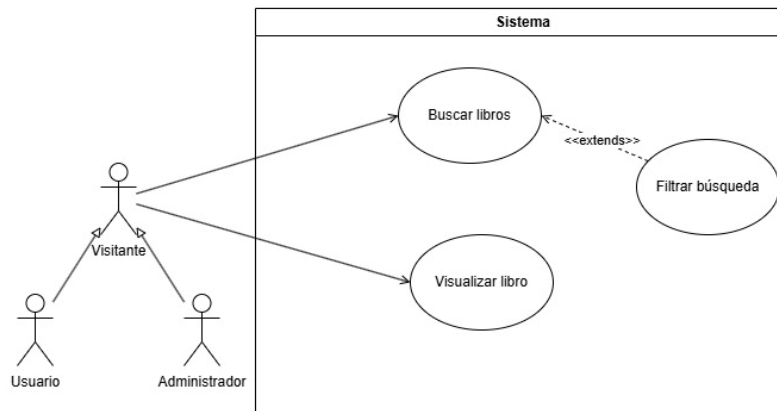


Figura 2. Diagrama de casos de uso para la búsqueda y visualización de libros.

En la Figura 2 se muestra el caso de uso referente a la búsqueda y visualización de libros, la especificación de este caso de uso se explica en la Tabla 3, Tabla 4 y Tabla 5.

ID	BKS-1	
Nombre	Buscar libros	
Actores	Visitante, Usuario, Administrador	
Descripción	Permite a los actores buscar libros en el sistema mediante palabras clave o criterios específicos.	
Precondición	• Debe existir una base de datos con libros registrados. • El sistema debe estar en funcionamiento.	
Flujo principal	Paso	Acción
	1.	El actor accede a la opción de búsqueda.



Instituto Politécnico Nacional
Escuela Superior de Cómputo



	2.	Ingresa una palabra clave o criterio de búsqueda.
	3.	El sistema valida la información ingresada.
	4.	El sistema procesa la solicitud y muestra los resultados encontrados.
Postcondiciones	Se muestra una lista de libros que coinciden con la búsqueda realizada.	
Flujo alterno	Paso	Acción
	1.	Si no hay coincidencias, el sistema muestra un mensaje indicando que no se encontraron libros.
Comentarios:		

Tabla 3. Especificación del caso de uso para la búsqueda de un libro.

ID	BKS-2	
Nombre	Filtrar búsqueda	
Actores	Visitante, Usuario, Administrador	
Descripción	Permite a los actores refinar los resultados de la búsqueda utilizando filtros avanzados.	
Precondición	• El actor debe haber realizado una búsqueda previamente. • Debe haber resultados que puedan ser filtrados.	
Flujo principal	Paso	Acción
	1.	El actor selecciona la opción de filtrar resultados.
	2.	Elige los criterios de filtrado (autor, año, categoría, etc.).
	3.	El sistema aplica los filtros y muestra los resultados actualizados.



Instituto Politécnico Nacional
Escuela Superior de Cómputo



Postcondiciones	Se muestra una lista refinada de libros que cumplen con los filtros aplicados.	
Flujo alterno	Paso	Acción
	1.	Si ningún libro coincide con los filtros seleccionados, el sistema muestra un mensaje de advertencia.
Comentarios:		

Tabla 4. Especificación del caso de uso para el filtrado de los libros.

ID	BKS-3	
Nombre	Visualizar libro	
Actores	Visitante, Usuario, Administrador	
Descripción	Permite a los actores acceder a la información detallada de un libro.	
Precondición	• Debe haber libros registrados en el sistema. • El actor debe haber encontrado un libro en la búsqueda.	
Flujo principal	Paso	Acción
	1.	El actor selecciona un libro de la lista de resultados.
	2.	El sistema muestra la información detallada del libro (título, autor, descripción, etc.).
Postcondiciones	El actor puede visualizar la información del libro seleccionado.	
Flujo alterno	Paso	Acción
	1.	Si ocurre un error al recuperar la información, el sistema muestra un mensaje de error y permite al usuario intentarlo nuevamente.
Comentarios:		

Tabla 5. Especificación del caso de uso para visualizar un libro en específico.

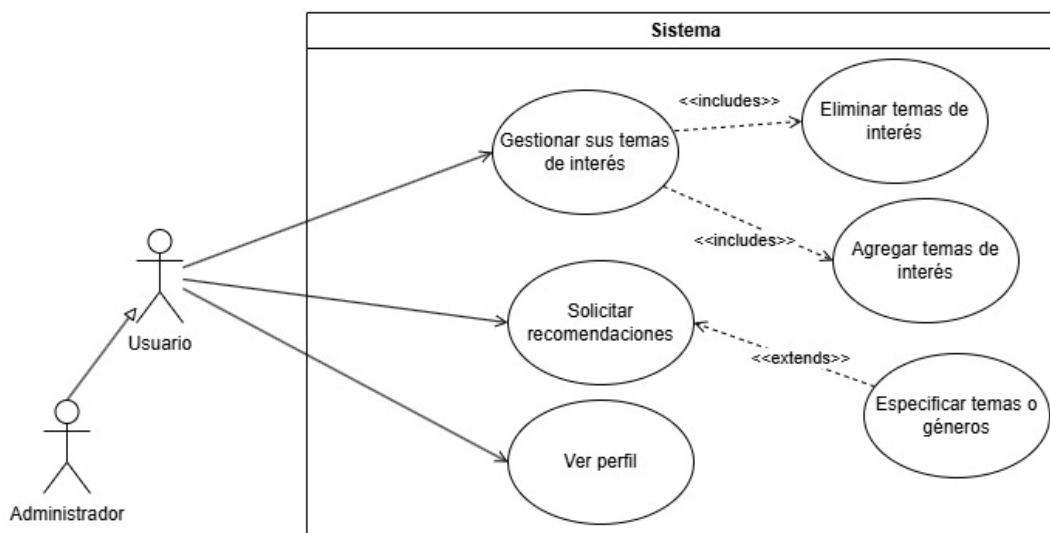


Figura 3. Diagrama de casos de uso para las acciones de un usuario autenticado.

En la Figura 3 se muestran los casos de uso para un usuario autenticado, así como la relación de herencia entre usuarios. En las tablas Tabla 6, Tabla 7 y Tabla 8 se muestra la especificación de casos de uso de los presentados en el diagrama.

ID	BKS-4	
Nombre	Gestionar sus temas de interés	
Actores	Usuario, Administrador	
Descripción	Permite al usuario administrar los temas de interés para recibir recomendaciones personalizadas.	
Precondición	El usuario debe haber iniciado sesión en el sistema.	
Flujo principal	Paso	Acción
	1.	El usuario accede a la opción de gestión de temas de interés.
	2.	El sistema muestra la lista actual de temas seleccionados.
	3.	El usuario elige modificar los temas de interés (agregar o eliminar).



Instituto Politécnico Nacional
Escuela Superior de Cómputo



	4.	El sistema actualiza la lista de temas y confirma los cambios.
Postcondiciones	La lista de temas de interés del usuario se actualiza correctamente.	
Flujo alternativo	Paso	Acción
	1.	Si ocurre un error al modificar los temas, el sistema muestra un mensaje de error.
Comentarios:		

Tabla 6. Especificación de caso de uso para gestionar los temas de interés de un usuario.

ID	BKS-5	
Nombre	Ver perfil	
Actores	Usuario, Administrador	
Descripción	Permite al usuario acceder a la información de su perfil dentro del sistema.	
Precondición	El usuario debe haber iniciado sesión.	
Flujo principal	Paso	Acción
	1.	El usuario accede a la opción de ver perfil.
	2.	El sistema recupera y muestra la información del usuario.
Postcondiciones	El usuario visualiza la información de su perfil.	
Flujo alternativo	Paso	Acción
	1.	Si ocurre un error, el sistema muestra un mensaje informando la falla.
Comentarios:		

Tabla 7. Especificación de caso de uso para la visualización del perfil de un usuario.

ID	BKS-6
Nombre	Solicitar recomendaciones
Actores	Usuario, Administrador



Instituto Politécnico Nacional
Escuela Superior de Cómputo



Descripción	Permite al usuario recibir recomendaciones basadas en sus temas de interés.	
Precondición	• El usuario debe haber iniciado sesión. • Debe tener al menos un tema de interés seleccionado.	
Flujo principal	Paso	Acción
	1.	El usuario accede a la opción de solicitar recomendaciones.
	2.	El sistema analiza los temas de interés del usuario.
	3.	Se generan y muestran las recomendaciones basadas en los temas seleccionados.
Postcondiciones	Se presentan al usuario las recomendaciones generadas.	
Flujo alternativo	Paso	Acción
	1.	El sistema informa al usuario que debe agregar temas antes de recibir recomendaciones.
Comentarios:		

Tabla 8. Especificación del caso de uso para la recomendación de libros.

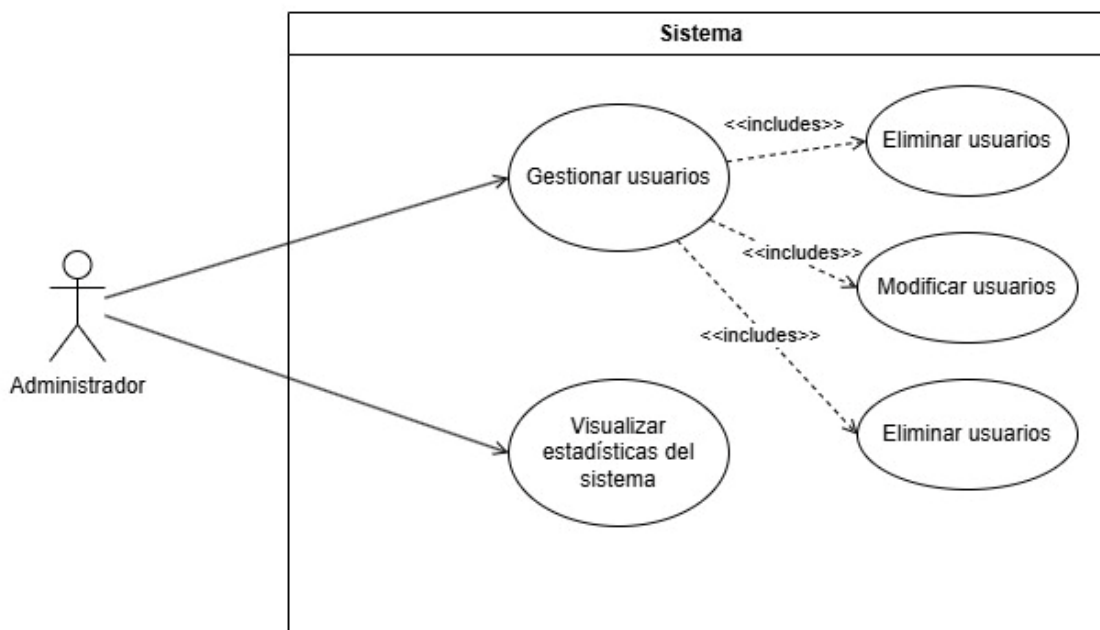


Figura 4. Diagrama de casos de uso para un administrador.

En la Figura 4 se muestra el diagrama de casos de uso para un actor administrador, mostrando las acciones que puede realizar. En las tablas Tabla 9 y Tabla 10 se especifican los casos de uso más importantes de este.

ID	ADM-1	
Nombre	Gestionar Usuarios	
Actores	Administrador	
Descripción	Permite al Administrador realizar operaciones de gestión sobre los usuarios del sistema, como eliminación y modificación.	
Precondición	• El Administrador debe haber iniciado sesión en el sistema. • El sistema debe tener usuarios registrados (para eliminar o modificar).	
Flujo principal	Paso	Acción



Instituto Politécnico Nacional
Escuela Superior de Cómputo



	1.	El Administrador selecciona la opción "Gestionar usuarios".
	2.	El sistema muestra las opciones disponibles (eliminar o modificar usuarios).
	3.	El Administrador selecciona la acción deseada (eliminar o modificar).
	4.	El sistema ejecuta la acción y confirma su realización.
Postcondiciones	- Los cambios realizados (eliminación o modificación) se reflejan en el sistema. - Se registra la acción en el historial del sistema.	
Flujo alternativo	Paso	Acción
	1.	Si ocurre un error, el sistema muestra un mensaje informando la falla.
Comentarios:		

Tabla 9. Especificación de caso de uso para la administración de usuarios

ID	ADM-2	
Nombre	Visualizar Estadísticas del Sistema	
Actores	Administrador	
Descripción	Permite al Administrador acceder a información estadística relevante del sistema.	
Precondición	- El Administrador debe haber iniciado sesión en el sistema. - El sistema debe tener datos generados para mostrar estadísticas.	
Flujo principal	Paso	Acción
	1.	El Administrador selecciona la opción "Visualizar estadísticas del sistema".



	2.	El sistema recopila y muestra las estadísticas solicitadas.
	3.	El Administrador revisa la información.
Postcondiciones	El Administrador obtiene la información estadística requerida.	
Flujo alterno	Paso	Acción
	1.	El sistema informa al usuario que debe agregar temas antes de recibir recomendaciones.
Comentarios:		

Tabla 10. Especificación de caso de uso para la visualización de estadísticas.

Caso de uso principal

El caso de uso principal es la búsqueda de libros, debido a que a partir de este se desencadenan otros casos de usos útiles para la funcionalidad principal del sistema. Este caso de uso será la primer funcionalidad del sistema sin considerar las funcionalidades del administrados y las de inicio de sesión y registro.



Instituto Politécnico Nacional

Escuela Superior de Cómputo

Guía técnica





En esta sección, se describen las herramientas tecnológicas utilizadas para el proyecto, describiendo a detalle su aportación y a su vez, dando una guía de instalación del proyecto.

Herramientas utilizadas

Las herramientas utilizadas en el desarrollo de este proyecto son:

Backend

- **Java con Spring boot:** Se utilizó la versión 21 de Java en conjunto del framework de spring boot, con estas herramientas se realizará la API REST del sistema, siendo de utilidad estas herramientas por el desempeño que dan, la integración de otras herramientas útiles como spring security, y el soporte con el que cuentan estas herramientas.
- **Spring security:** Una biblioteca del framework de spring boot, es de utilidad para el manejo de roles y permisos en la aplicación, así como brindar herramientas de cifrado.
- **JWT:** Json Web Token es una tecnología que encripta información que está en formato JSON, dando así la facilidad de manejar las sesiones mediante este token.
- **Gradle:** Gradle es el gestor de paquetes para manejar las bibliotecas necesarias, así como define las estrategias para compilar y construir el proyecto.

Frontend

- **Vite:** Vite es un framework para aplicaciones con React JS, siendo útil por su optimización del código de producción, así como proporcionar herramientas extra.
- **React JS:** Útil para la creación de aplicaciones web reactivas, dando una gran facilidad para la integración de un servidor genérico con la aplicación web, además de contar con un buen desempeño.



- **TypeScript:** A diferencia de Javascript, TypeScript permite crear código de JS con un tipado fuerte, manteniendo un mejor control de las funciones e integraciones utilizadas.
- **Axios:** Axios es una biblioteca de javascript que permite hacer solicitudes GET, POST, PUT, DELETE a un servidor, de forma sencilla.
- **Tailwind CSS:** Esta tecnología es un framework de CSS que permite agregar código de CSS dentro de las etiquetas HTML sin necesidad de tener que contar con un archivo CSS aparte, además de dar un mejor desempeño a la aplicación al solo cargar los estilos necesarios.

Base de datos

- **PostgreSQL:** Postgres es una base de datos relacional que permite almacenar la información que sea utilizada en el sistema, se optó por postgres por la gran variedad de implementaciones gracias a que es de código abierto.

DevOps

- **Docker:** Docker es una herramienta para el manejo de contenedores, siendo de utilidad para crear entornos aislados de cada programa del sistema, brindando también facilidad para mantener activo el sistema.
- **Docker-compose:** Esta herramienta es una forma declarativa mediante archivos en formato YAML para crear contenedores, siendo más sencillo la declaración de las características del contenedor en lugar de hacerlo mediante comandos.

Control de versiones

- **Git:** Git es la herramienta que sirve para llevar un correcto control de versiones, teniendo una gran forma de optimizar el tamaño de todos los cambios, así como contar con implementaciones extra que sirven para el trabajo colaborativo.



- **GitHub:** Es la plataforma remota o en la nube que almacena el código fuente del sistema, además que cuenta con herramientas útiles para la creación del sistema.

Configuración de la base de datos

La aplicación Graph News utiliza PostgreSQL como sistema de gestión de base de datos relacional. La configuración busca implementar buenas prácticas de seguridad para el manejo de credenciales de accesos empleando variables de entorno y un sistema de entidades JPA.

URL de Conexión

- **Propiedad:** `spring.datasource.url=${DATABASE_URL}`
- **Descripción:** Define la URL de conexión a la base de datos PostgreSQL.
- **Implementación:** Utiliza una variable de entorno `DATABASE_URL` que típicamente sigue el formato `jdbc:postgresql://<host>:<port>/<database_name>`.
- **Ejemplo en desarrollo:** `jdbc:postgresql://db:5432/crud` (según el `docker-compose.yml`)

Credenciales de Acceso

- **Propiedades:**
 - `spring.datasource.username=${DATABASE_USER}`
 - `spring.datasource.password=${DATABASE_PASSWORD}`
- **Descripción:** Establecen el usuario y contraseña para autenticarse con la base de datos.
- **Implementación:** Utilizan variables de entorno para evitar hardcodear credenciales en el código.

Configuración de JPA/Hibernate

- **Propiedad:** `spring.jpa.hibernate.ddl-auto=update`
- **Descripción:** Controla cómo Hibernate maneja el esquema de base de datos.



- **Comportamiento:** Con el valor update, Hibernate:
 - Actualiza automáticamente el esquema cuando detecta cambios en las entidades
 - Crea tablas si no existen
 - No elimina datos existentes ni tablas no referenciadas
 - Adecuado para entornos de desarrollo, pero debe usarse con precaución en producción

Driver y Dialecto

- **Propiedad:** `spring.datasource.driver-class-name=org.postgresql.Driver`
- **Descripción:** Especifica el driver JDBC para PostgreSQL.
- **Propiedad:** `spring.jpa.database-platform = org.hibernate.dialect.PostgreSQLDialect`
- **Descripción:** Define el dialecto SQL que Hibernate debe usar.
- **Importancia:** Permite a Hibernate generar SQL optimizado específicamente para PostgreSQL.

Configuración del sistema de seguridad (Spring Security)

Sistema usado por medio de JWT

Dentro del archivo `JWTAuthenticationFilter.java` se define un filtro ejecutado una vez por solicitud hacia la validación de un token

Servicio de detalles de usuario

La clase `UserDetailsServiceImplementation.java` extiende la interfaz estándar de Spring Security que permite que el sistema de seguridad trabaje directamente con las entidades de la aplicación. Algunas de las facilidades que nos proporciona este servicio son:

- Cargar información detallada del usuario desde la base de datos



- Transformar el modelo de dominio User en el objeto UserDetails que Spring Security reconoce
- Facilitar la verificación de credenciales durante el proceso de autenticación

Unidades de la gestión JWT

La utilidad de JWT, se da por medio del archivo JwtUtil.java, que contiene métodos para generar y validar tokens JWT, así como para extraer información de estos, ofreciendo algunos métodos para:

- Generar nuevos tokens con información personalizada (claims)
- Validar la autenticidad y vigencia de los tokens existentes
- Extraer información específica contenida dentro de los tokens
- Manejar la firma criptográfica de los tokens mediante algoritmos seguros

Configuraciones de Seguridad.

La configuración de seguridad se define en el archivo SecurityConfig.java, pues aquí se especifican las reglas de autorización para los endpoints de acceso público, acceso para administradores y los accesos para el servicio de autenticación:

Roles, permisos y endpoints protegidos

Aquí, se definen los diferentes roles y endpoints disponibles en el sistema. Así mismo, se especifica su verbo de petición, una breve descripción y el tipo de acceso que este tiene que puede ser de tipo público (para cualquier usuario), autenticado (requiere de una sesión en el sistema) o de tipo ADMIN (solamente usuarios registrados con el rol de 'ADMIN' pueden acceder).

Roles:

- **USER:** Usuario regular.
- **ADMIN:** Administrador del sistema.

Endpoints:

En esta sección, se detallarán los endpoints creados en la API REST del programa.



URL: /api/user/{id}

Método HTTP: GET

Parámetros:

- ID: Colocado en la URL, indica el ID del usuario del que queremos obtener su información.
- Token: El token es el JWT, este debe estar en la cabecera de autorización.

Respuestas:

- Token inválido. (HTTP 400)
- Token expirado. (HTTP 403)
- No se tiene permiso. (HTTP 401)
- Éxito. (HTTP 200)

Ejemplo de solicitud (body): N/A.

Ejemplo de respuesta (body):

```
1. {  
2.   message: 'Success',  
3.   user: {  
4.     id: 1,  
5.     name: 'Frijolito Hernandez',  
6.     email: 'Frijolito@gmail.com',  
7.     role: 'USER'  
8.   },  
9. }  
10.
```

URL: /api/user/register

Método HTTP: POST

Parámetros:

- Name: Es el nombre del usuario que se quiere registrar.
- Email: Es el correo del usuario que se quiere registrar.
- Password: Es la contraseña del usuario para acceder al sistema.



Respuestas:

- El correo ya está en uso. (HTTP 400)
- Éxito. (HTTP 200)

Ejemplo de solicitud (body):

```
1. {  
2.   name: 'Frijolito Hernandez',  
3.   email: 'Frijolito@gmail.com',  
4.   password: '12345678'  
5. }
```

Ejemplo de respuesta (body):

```
1. {  
2.   message: 'Success',  
3.   user: {  
4.     id: 1,  
5.     name: 'Frijolito Hernandez',  
6.     email: 'Frijolito@gmail.com',  
7.     role: 'USER'  
8.   },  
9.   token: '...'  
10. }
```

URL: /api/user/login

Método HTTP: POST

Parámetros:

- Email: Es el correo del usuario con el que se quiere acceder.
- Password: Es la contraseña asociada al correo indicado.

Respuestas:

- Credenciales incorrectas. (HTTP 400)
- Éxito. (HTTP 200)

Ejemplo de solicitud (body):

```
1. {  
2.   email: 'Frijolito@gmail.com',  
3.   password: '12345678'  
4. }
```

Ejemplo de respuesta (body):



```
1. {  
2.   message: 'Success',  
3.   user: {  
4.     id: 1,  
5.     name: 'Frijolito Hernandez',  
6.     email: 'Frijolito@gmail.com',  
7.     role: 'USER'  
8.   },  
9.   token: '...'  
10. }  
11.
```

URL: /api/user/all

Método HTTP: GET

Parámetros:

- Token: El token es el JWT, este debe estar en la cabecera de autorización.

Respuestas:

- Token inválido. (HTTP 400)
- Token expirado. (HTTP 403)
- No se tiene permiso. (HTTP 401)
- Éxito. (HTTP 200)

Ejemplo de solicitud (body): N/A

Ejemplo de respuesta (body):

```
1. {  
2.   message: 'Success',  
3.   users: [  
4.     {  
5.       id: 1,  
6.       name: 'Frijolito Hernandez',  
7.       email: 'Frijolito@gmail.com',  
8.       role: 'USER'  
9.     },  
10.    {  
11.      id: 2,  
12.      name: 'Pakis Jimenez',  
13.      email: 'pakis@gmail.com',  
14.      role: 'ADMIN'  
15.    }  
16.  ]  
17. }
```



Instituto Politécnico Nacional
Escuela Superior de Cómputo



URL: /api/user/{id}

Método HTTP: DELETE

Parámetros:

- ID: Colocado en la URL, indica el ID del usuario que buscamos borrar.
- Token: El token es el JWT, este debe estar en la cabecera de autorización.

Respuestas:

- Token inválido. (HTTP 400)
- Token expirado. (HTTP 403)
- No se tiene permiso. (HTTP 401)
- El correo ya está usado. (HTTP 409)
- Entidad no encontrada. (HTTP 410)
- Éxito. (HTTP 200)

Ejemplo de solicitud (body): N/A

Ejemplo de respuesta (body):

```
1. {  
2.   message: 'Success',  
3. }
```

URL: /api/user/{id}

Método HTTP: PUT

Parámetros:

- Name: Nombre nuevo del usuario.
- Email: Correo nuevo del usuario.
- Password: La contraseña nueva del usuario.
- ID: Colocado en la URL, indica el ID del usuario que buscamos borrar.
- Token: El token es el JWT, este debe estar en la cabecera de autorización.



Respuestas:

- Token inválido. (HTTP 400)
- Token expirado. (HTTP 403)
- No se tiene permiso. (HTTP 401)
- El correo ya está usado. (HTTP 409)
- Entidad no encontrada. (HTTP 410)
- Éxito. (HTTP 200)

Ejemplo de solicitud (body):

```
1. {  
2.   name: 'Frijolito Hernandez',  
3.   email: 'Frijolito@gmail.com',  
4. }
```

Ejemplo de respuesta (body):

```
1. {  
2.   message: 'Success',  
3.   user: {  
4.     id: 1,  
5.     name: 'Frijolito Hernandez',  
6.     email: 'Frijolito@gmail.com',  
7.     role: 'USER'  
8.   }  
9. }
```

Permisos

Acceso público: Estos únicamente incluyen los endpoints para efectuar solicitudes sin una autenticación necesariamente, así como la obtención de tokens.

- Registro de nuevo usuarios (POST /api/user/signup)
- Acceso al sistema (POST/api/user/login)

Acceso administrativo: Las operaciones de gestión de usuarios como:

- Listado de todos los usuarios (GET /api/user/all)
- Eliminación de usuarios (DELETE /api/user/**)
- Modificación de usuarios (PUT /api/user/**) están restringidas exclusivamente a usuarios con rol ADMIN.



Acceso autenticado: Todos los demás endpoints de la aplicación requieren que el usuario esté autenticado, independientemente de su rol específico.

Archivos relacionados

En esta sección se listan los archivos relacionados que no se involucran directamente con las funcionalidades del negocio.

application.properties

El archivo application properties contiene configuraciones principales para la aplicación Spring Boot, como la configuración de la base de datos, el puerto del servidor, y las propiedades de seguridad JWT, por ende, tiene por tarea el definir cómo la aplicación se conecta a la base de datos y maneja la autenticación y autorización.

Conexión a la base de datos:

- Se emplean variables de entorno como: (`${DATABASE_URL}`, `${DATABASE_USER}`, `${DATABASE_PASSWORD}`) para las credenciales de PostgreSQL.
- Se configura el Driver para PostgreSQL con `spring.datasource.driver-class-name=org.postgresql.Driver`.
- Dialecto Hibernate para trabajar con PostgreSQL en `spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect`.
- La propiedad `spring.jpa.hibernate.ddl-auto=update` permite actualizar el esquema automáticamente.

Puerto del Servidor:

- Configurado dinámicamente con `server.port=${PORT}`.

Seguridad JWT:

- Secret key y tiempo de expiración configurados mediante variables de entorno.
- `security.jwt.secret=${JWT_SECRET}` para la clave secreta de firma.



- `security.jwt.expiration=${JWT_EXPIRATION}` para el tiempo de validez del token.

`docker-compose.yml`

El archivo `Docker-compose.yml` tiene por función el definir los servicios Docker requeridos para la aplicación, incluyendo la base de datos PostgreSQL y el backend de la aplicación, correspondiendo al entorno de producción.

- Las variables de entorno se definen en `docker-compose.yml` para el entorno de desarrollo.
- Incluye credenciales de base de datos, puerto, secret JWT y tiempo de expiración.

`build.gradle`

El archivo `build.gradle.kts` define las dependencias y configuraciones de Gradle para el proyecto backend, este incluye las dependencias necesarias para Spring Boot, seguridad, JWT, y PostgreSQL, así como configuraciones específicas de compilación y pruebas, gestionando las bibliotecas y plugins que la aplicación necesita para funcionar.

`SecurityConfig.java`

`SecurityConfig.java` se encarga de configurar la seguridad de Spring Security, incluyendo la configuración de los filtros JWT y las reglas de autorización, es decir, define qué endpoints son accesibles para cada rol y cuáles requieren autenticación, asegurando que solo los usuarios autorizados puedan acceder a ciertas partes de la aplicación.

Autenticación y Autorización:

- Implementa `SecurityFilterChain` para definir reglas de acceso.
- Se utiliza CSRF en modo deshabilitado a través de la instrucción: `csrf(AbstractHttpConfigurer::disable)`.



Instituto Politécnico Nacional
Escuela Superior de Cómputo



- Se configura CORS para permitir solicitudes desde `http://localhost:5173`, esto para que la aplicación cliente (react) pueda consumir los endpoints definidos.

Reglas de Acceso por Endpoint:

- Endpoints públicos: `POST /api/user/signup` y `POST /api/user/login`.
- Endpoints restringidos para administradores:
 - `GET /api/user/all`
 - `DELETE /api/user/**`
 - `PUT /api/user/**`
- Todas las demás rutas requieren autenticación.

Integración JWT:

- Se agrega `JWTAuthenticationFilter` en la cadena de filtros antes de `UsernamePasswordAuthenticationFilter`.

Encriptación de Contraseñas:

- Se utiliza `BCryptPasswordEncoder` para el hashing seguro de contraseñas.

Gestor de Autenticación:

- Se configura un 'bean' `AuthenticationManager` personalizado que utiliza el servicio de detalles de usuario y el codificador de contraseñas para gestionar las autenticaciones del usuario.

JwtUtil.java

Este archivo se encarga de la generación, validación y extracción de la información de tokens firmados con HMAC-SHA256 que incluye los claims y detalles como el email de los usuarios registrados, las fechas en que se crean y expiran los tokens. Además, incluye métodos para extraer información específica de los usuarios.

Generación de Tokens:



Instituto Politécnico Nacional
Escuela Superior de Cómputo



- generateToken(): Se utiliza para crea un token con claims, email del usuario, fecha de emisión y expiración.
 - Utiliza HMAC-SHA256 para firmar los tokens.

Validación de Tokens:

- isValidToken(): Verifica que el email en el token coincida y que no haya expirado.
- isTokenExpired(): Comprueba si la fecha de expiración es anterior a la actual.

Extracción de Información:

- extractClaim(): Permite extraer claims específicos del token.
- extractAllClaims(): Obtiene todos los claims del token.

Configuración de Seguridad:

- Secret key y tiempo de expiración se inyectan desde las propiedades de la aplicación, definidas en el archivo: "application.properties"
- La clave se codifica en Base64 y se utiliza para crear un SecretKeySpec.

CORS (Cross-Origin Resource Sharing)

Se implementa directamente en SecurityConfig.java para permitir el intercambio de recursos entre el backend y el frontend.

- **Accesos:** Permite solicitudes desde http://localhost:5173 (origen del frontend en desarrollo).
- **Métodos permitidos:** GET, POST, PUT, DELETE.
- **Cabeceras:** Permite todas las cabeceras con `setAllowedHeaders(List.of("*"))`.
- **Credenciales:** Habilita las credenciales con `setAllowCredentials(true)`.

Variables de Entorno

Docker-Compose:

- Se definen en docker-compose.yml para el entorno de desarrollo.



Instituto Politécnico Nacional
Escuela Superior de Cómputo



- Incluye credenciales de base de datos, puerto, secret JWT y tiempo de expiración.

Application Properties:

- En el archivo application.properties referencia las variables de entorno con la sintaxis \${VARIABLE}.

Seguridad:

- JWT Secret en base64:
c2VjcmV0S2V5MTIzNDU2Nzg5MDEyMzQ1Njc4OTAxMjM0NTY3ODkwMTIzNA.
zNA.
- Tiempo de expiración configurado en milisegundos (24 horas: 86400000).

Contenerización:

- El Dockerfile construye la aplicación y la ejecuta sin exponer directamente las variables sensibles en la imagen.



Instituto Politécnico Nacional

Escuela Superior de Cómputo

Apéndices



ESCOM



Apéndice A: Glosario de términos

API: Application Programming Interface, conjunto de definiciones y protocolos para construir e integrar software.

Entidad: Objeto o concepto del mundo real que puede ser identificado y representado (ej. persona, lugar, organización).

JWT: JSON Web Token, estándar para la creación de tokens de acceso.

Lematización: Proceso de reducir una palabra a su forma base o raíz.

Relación: Conexión o asociación entre dos o más entidades.